

Azure Data Engineer

7 × Realistic 1-Hour Mock Interviews

TechWithChandrea

Conversational interview practice with detailed answers, realistic follow-ups, production scenarios and professional closing.

What's Inside

Interview 1 — Azure Data Engineering Fundamentals + Project

Target experience: 3-5 years • Approx. 1 hour

Interview 2 — ADF + Incremental Loading + ETL

Target experience: 4-6 years • Approx. 1 hour

Interview 3 — Databricks + PySpark + Delta

Target experience: 4-7 years • Approx. 1 hour

Interview 4 — SQL + Data Modeling + Performance

Target experience: 4-7 years • Approx. 1 hour

Interview 5 — Production Support + Troubleshooting

Target experience: 5-8 years • Approx. 1 hour

Interview 6 — End-to-End Project + Architecture

Target experience: 5-8 years • Approx. 1 hour

Interview 7 — Senior Final Mock Interview

Target experience: 6-10 years • Approx. 1 hour

Each interview follows a realistic flow: introduction → project discussion → technical deep dive → follow-up questions → real-world scenarios → candidate questions → interview closing.

Interview 1

Azure Data Engineering Fundamentals + Project

Target experience: 3-5 years • Approx. 1 hour

Interviewer: Can you walk me through your end-to-end architecture?

Candidate: The typical flow is source systems → ADF ingestion → ADLS Gen2 raw layer → Databricks transformations → Delta curated layer → serving/reporting. I keep ingestion and transformation responsibilities separate so that raw data remains available for replay and troubleshooting. ADF handles orchestration and movement, while Databricks handles complex transformations and Spark processing. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Why did you choose ADLS Gen2?

Candidate: ADLS Gen2 is useful because hierarchical namespace gives the storage account a file-system-like directory structure and supports Azure RBAC together with POSIX-like ACLs. That makes it a strong storage layer for analytics workloads. I would still design access carefully using least privilege and security groups rather than giving broad permissions. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: ADF or Databricks — when would you use each?

Candidate: I use Azure Data Factory mainly for orchestration, scheduling, connectors, parameterization and data movement. I use Databricks when I need distributed Spark processing, complex transformations, PySpark, Delta Lake operations or large-scale data processing. They are complementary: ADF can orchestrate a Databricks job instead of trying to make one service do everything. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you implement incremental loading?

Candidate: I normally store a high-watermark value in a control table. ADF reads the previous successful watermark, extracts rows using a reliable increasing/change-detection column, loads the target, validates the result, and only then advances the stored watermark. The boundary condition must be designed carefully so records are not skipped or duplicated. In a real interview, I would then

give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What if the timestamp is not unique?

Candidate: If a timestamp is not unique, I would not assume timestamp-only filtering is safe. I would look for a reliable increasing key, CDC, change tracking, or a composite boundary such as timestamp plus key where the source semantics support it. I would test equal-timestamp records and retry scenarios because the boundary is where incremental pipelines commonly lose data. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What happens if a pipeline fails halfway through?

Candidate: I design the process to be restartable and idempotent. For example, I write to a staging area first, validate the data, and then merge or publish it into the target. If a retry occurs, the same batch should not create duplicates. The watermark is also updated only after successful completion. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you handle inserts and updates?

Candidate: If the source provides CDC, I use the operation type and keys to apply changes. For lakehouse targets, Delta MERGE is useful because it can match records by business key and perform updates and inserts in one operation. Deletes need to be handled explicitly based on the source's change semantics. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you secure secrets?

Candidate: I avoid hardcoding credentials in notebooks or pipelines. I prefer managed identities where supported and Azure Key Vault for secrets that genuinely need secret storage. Access is controlled through least-privilege permissions and environment-specific configuration. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you monitor ADF pipelines?

Candidate: I monitor pipeline and activity runs, duration, failures, retry behavior and data-volume indicators. For production, I also want alerts for failed pipelines, SLA breaches and unexpected row counts. Logs should contain enough context to identify the pipeline, batch, source and target involved. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What is the purpose of raw, curated and serving layers?

Candidate: The raw layer preserves source data with minimal transformation. The curated layer applies cleansing, standardization and business transformations. The serving layer is optimized for consumption by reports, applications or downstream users. Separating layers improves traceability and makes replay and troubleshooting easier. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you handle schema changes?

Candidate: First I determine whether the change is compatible or breaking. Adding a nullable column can often be handled through controlled schema evolution, while changing a data type or removing a column requires impact analysis. I prefer detecting unexpected schema changes early rather than allowing them to silently corrupt downstream data. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What is Delta Lake and why use it?

Candidate: Delta Lake adds a transaction layer over lake storage and provides capabilities such as ACID transactions, schema enforcement and evolution features, table history and MERGE. That makes lakehouse tables easier to operate reliably than treating a folder of Parquet files as an unmanaged table. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How would you improve a slow Spark job?

Candidate: I first identify where time is being spent rather than blindly adding resources. I check the Spark UI for shuffles, skew, large scans and long-running stages. Then I reduce unnecessary columns and rows, improve partition pruning, optimize joins, handle skew and use appropriate cluster sizing. I would support that decision with evidence from the actual workload, such as execution-plan information, Spark UI metrics, row counts or runtime comparisons. After the change, I would

verify correctness as well as performance.

Interviewer: What is a shuffle?

Candidate: A shuffle redistributes data between Spark partitions, commonly during joins, groupBy and aggregations. It can be expensive because data may move across the network and require serialization and disk I/O. Reducing unnecessary shuffles is therefore an important performance optimization. I would support that decision with evidence from the actual workload, such as execution-plan information, Spark UI metrics, row counts or runtime comparisons. After the change, I would verify correctness as well as performance.

Interviewer: How do you test pipelines?

Candidate: I test both functional and data-quality behavior. I validate row counts, nulls, duplicates, key relationships, incremental boundaries and error handling. I also test reruns and failure scenarios because a pipeline that works only on the first successful run is not production-ready. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you deploy ADF changes?

Candidate: I develop in the Git-connected development factory, use branches and pull requests for review, then promote the published version to UAT and production through CI/CD. Environment-specific values are parameterized so production is not manually edited into a different version of the pipeline. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: A production pipeline is suddenly slow. What do you check first?

Candidate: I compare the current run with previous successful runs and identify which activity or stage changed. Then I check source volume, query performance, file counts, Spark stages, cluster capacity and external service latency. I want evidence before changing configuration. I would support that decision with evidence from the actual workload, such as execution-plan information, Spark UI metrics, row counts or runtime comparisons. After the change, I would verify correctness as well as performance.

Interviewer: What does idempotency mean in data engineering?

Candidate: It means running the same logical batch more than once produces the same intended final state instead of duplicating data. I achieve this through deterministic batch identifiers, staging, merge logic, deduplication and careful

checkpoint or watermark management. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Tell me about a performance improvement you made.

Candidate: I would explain the original SLA, the bottleneck, the evidence I collected, the change I made and the measurable result. For example, if a transformation repeatedly scanned unnecessary columns and caused large shuffles, I could reduce the projection, improve filtering and optimize joins, then compare runtime and resource usage before and after. I would support that decision with evidence from the actual workload, such as execution-plan information, Spark UI metrics, row counts or runtime comparisons. After the change, I would verify correctness as well as performance.

Interviewer: How would you explain your project to a non-technical stakeholder?

Candidate: I focus on business flow rather than tools. I explain where the data comes from, how quickly it becomes available, how quality is controlled and what business decision it supports. I mention technologies only when they help explain reliability, scale or delivery speed. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What would you improve in your current platform?

Candidate: I would look for improvements in observability, reusable ingestion, data-quality automation, cost visibility and deployment consistency. I would prioritize changes based on production pain, business impact and engineering effort rather than introducing tools simply because they are newer. Natural transition into closing In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Good. I think we have covered the main technical areas.

Let's wrap up with a couple of questions from you. Interviewer: Alright, that covers my technical questions. Before we wrap up, do you have any questions for me?

Candidate: Yes, I have a couple of questions. Could you please tell me a little about the team I would be working with and the type of Azure data engineering challenges the team is currently handling? In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Sure. The team works on a mix of data platform development, production support and new data initiatives. We are currently focusing on improving reliability, data quality and scalability.

Candidate: Thank you. One more question — what would you expect from someone joining this role in the first three to six months? I would separate immediate recovery from permanent prevention: first restore the expected service or data safely, then document the root cause, contributing factors and preventive action.

Interviewer: We would expect the person to understand the platform, take ownership of assigned work, contribute to production support and gradually take responsibility for larger technical problems.

Candidate: That sounds good. Thank you for your time. I really enjoyed discussing the project, architecture and technical challenges with you. I would separate immediate recovery from permanent prevention: first restore the expected service or data safely, then document the root cause, contributing factors and preventive action.

Real Interview Closing

Interviewer: Alright, that covers my technical questions. Before we wrap up, do you have any questions for me?

Candidate: Yes, I have a couple. Could you tell me a little about the team I would be working with and the main Azure data-engineering challenges the team is currently solving?

Interviewer: Sure. The team works across data platform development, production support and new data initiatives, with a focus on reliability, quality and scalability.

Candidate: Thank you. One more question: what would you expect from someone joining this role during the first three to six months?

Interviewer: We would expect the person to understand the platform, take ownership of assigned work, contribute to production support and gradually take responsibility for larger technical problems.

Candidate: That sounds good. Thank you for your time. I really enjoyed discussing the architecture, project decisions and production scenarios with you.

Interviewer: Thank you for your time as well. We'll get back to you regarding the next steps.

Interviewer checklist

✓ Project clarity • ✓ Technical reasoning • ✓ Production thinking • ✓ Troubleshooting • ✓ Performance • ✓ Security • ✓ Communication • ✓ Candidate questions

Interview 2

ADF + Incremental Loading + ETL

Target experience: 4-6 years • Approx. 1 hour

Interviewer: Let's say you need to ingest 100 source tables. Would you create 100 separate pipelines?

Candidate: I would avoid unnecessary duplication. I would build a metadata-driven framework where the control table stores source table, target path, load type, watermark column, primary key and other configuration. A generic pipeline can read the metadata and execute the appropriate copy or transformation logic. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you decide when to update a watermark?

Candidate: Only after the corresponding batch has completed successfully and validation has passed. Updating it before the target is committed can cause missed records if the pipeline fails afterward. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What if records arrive late?

Candidate: I may use an overlap window, such as re-reading a small period, and then deduplicate or merge into the target. The exact window depends on source behavior. The goal is to protect against late-arriving records without creating duplicates. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What if there is no modified timestamp?

Candidate: I look for CDC, change tracking, an increasing key, source audit tables or another reliable change indicator. If none exists, a full comparison may be necessary, but I would evaluate the source size and performance before choosing that approach. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you make an ADF pipeline restartable?

Candidate: I use deterministic batch identifiers, staging locations and clear success criteria. Activities should be safe to retry, and the pipeline should avoid updating

control metadata until the target state is confirmed. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Pipeline parameter vs variable?

Candidate: Parameters are inputs supplied to a pipeline or dataset and generally remain fixed during that run. Variables are runtime values that can be changed while the pipeline executes. I use parameters for reusable configuration and variables for temporary execution state. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: A source file arrives late. What would you do?

Candidate: I would separate file arrival detection from processing. Depending on the SLA, I can use a scheduled check, event-based trigger or a retry/wait pattern. I also want the process to clearly report missing expected files rather than silently succeeding with incomplete data. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you prevent duplicate file processing?

Candidate: I maintain file-level metadata such as file name, size, checksum or source batch ID in an audit table. Before processing, I check whether that exact logical file or batch was already successfully processed. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you handle API pagination?

Candidate: I follow the API's documented pagination mechanism, such as next-link, page number or continuation token. I persist the state when necessary and make each page processing retryable. I also monitor rate limits and response completeness. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What if the API returns HTTP 429?

Candidate: 429 indicates rate limiting. I use controlled retries with backoff, respect Retry-After when provided, and avoid launching excessive parallel requests. I also monitor the API's documented limits. I would separate immediate recovery from

permanent prevention: first restore the expected service or data safely, then document the root cause, contributing factors and preventive action.

Interviewer: How do you handle errors in ADF?

Candidate: I capture failure details, route errors to an audit mechanism and alert the appropriate team for production failures. For transient errors I use limited retries, but I don't use retries to hide deterministic data-quality failures. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What audit fields would you keep?

Candidate: Typical fields include pipeline run ID, batch ID, source, target, start time, end time, status, rows read, rows written, error message and watermark values. These fields make operational troubleshooting much easier. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What if two pipeline runs overlap?

Candidate: I control concurrency where required and make the load logic safe even if overlap happens. For shared watermarks, I need a consistent locking or control strategy so two runs cannot incorrectly advance the same state. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you validate source and target counts?

Candidate: Counts are useful but not sufficient. I compare counts where appropriate and also validate keys, nulls, duplicates, sums or other business-level checks. A matching count can still contain incorrect records. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you handle nulls?

Candidate: I first understand whether null is valid business data or indicates an ingestion problem. I apply defaults only when the business rule requires them. Otherwise, I preserve the value and flag it through data-quality rules. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Explain idempotency with a practical example.

Candidate: If a batch containing 10,000 orders is retried, the target should still contain one logical copy of those orders. I can achieve that with a batch ID and merge or deduplication logic rather than blindly appending every retry. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How would you backfill six months of data?

Candidate: I would parameterize the date range and run controlled batches rather than modifying the normal incremental logic. I would monitor volume, validate each batch and protect production workloads from excessive concurrency. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: ADF is slow but Databricks is fast. Where would you investigate?

Candidate: I would identify the slow ADF activity first. The bottleneck may be source query performance, connector throughput, network limits, DIU configuration or file layout. I would inspect activity metrics before changing Databricks. I would support that decision with evidence from the actual workload, such as execution-plan information, Spark UI metrics, row counts or runtime comparisons. After the change, I would verify correctness as well as performance.

Interviewer: How do you design a metadata-driven pipeline?

Candidate: I keep configuration separate from orchestration logic. A control table defines source, target, load type, key columns, watermark and optional transformation settings. The generic pipeline reads the configuration and passes values into reusable activities. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you handle deletes?

Candidate: I need a reliable delete signal such as CDC, a source operation flag, or a source snapshot comparison. For a Delta target, I can apply deletes through supported MERGE logic or explicit delete operations based on the source semantics. I would not infer deletes merely because a record is absent from one incremental extract unless the source contract says that absence has that meaning. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Why use a control table?

Candidate: It centralizes operational metadata and allows one framework to handle many tables. It also provides a place to store last successful watermark, status, row counts and configuration. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Explain your incremental design in one minute.

Candidate: I store the last successful boundary in a control table, read it at the start, extract the next window, validate and load the target, and update the control state only after success. I also make the target operation idempotent and account for late-arriving records. Natural transition into closing In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Good. I think we have covered the main technical areas.

Let's wrap up with a couple of questions from you. Interviewer: Alright, that covers my technical questions. Before we wrap up, do you have any questions for me?

Candidate: Yes, I have a couple of questions. Could you please tell me a little about the team I would be working with and the type of Azure data engineering challenges the team is currently handling? In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Sure. The team works on a mix of data platform development, production support and new data initiatives. We are currently focusing on improving reliability, data quality and scalability.

Candidate: Thank you. One more question — what would you expect from someone joining this role in the first three to six months? I would separate immediate recovery from permanent prevention: first restore the expected service or data safely, then document the root cause, contributing factors and preventive action.

Interviewer: We would expect the person to understand the platform, take ownership of assigned work, contribute to production support and gradually take responsibility for larger technical problems.

Candidate: That sounds good. Thank you for your time. I really enjoyed discussing the project, architecture and technical challenges with you. I would separate immediate recovery from permanent prevention: first restore the expected service or data safely, then document the root cause, contributing factors and preventive action.

Real Interview Closing

Interviewer: Alright, that covers my technical questions. Before we wrap up, do you have any questions for me?

Candidate: Yes, I have a couple. Could you tell me a little about the team I would be working with and the main Azure data-engineering challenges the team is currently solving?

Interviewer: Sure. The team works across data platform development, production support and new data initiatives, with a focus on reliability, quality and scalability.

Candidate: Thank you. One more question: what would you expect from someone joining this role during the first three to six months?

Interviewer: We would expect the person to understand the platform, take ownership of assigned work, contribute to production support and gradually take responsibility for larger technical problems.

Candidate: That sounds good. Thank you for your time. I really enjoyed discussing the architecture, project decisions and production scenarios with you.

Interviewer: Thank you for your time as well. We'll get back to you regarding the next steps.

Interviewer checklist

✓ Project clarity • ✓ Technical reasoning • ✓ Production thinking • ✓ Troubleshooting • ✓ Performance • ✓ Security • ✓ Communication • ✓ Candidate questions

Interview 3

Databricks + PySpark + Delta

Target experience: 4-7 years • Approx. 1 hour

Interviewer: Why Databricks for data engineering?

Candidate: Databricks provides a managed Spark environment and integrates well with Azure storage and lakehouse technologies. It is useful for large-scale transformations, notebooks, jobs, Delta tables and collaborative data engineering. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What is lazy evaluation in Spark?

Candidate: Transformations build a logical execution plan but generally do not execute immediately. Spark waits until an action is called, allowing the optimizer to plan the work more efficiently. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Transformation vs action?

Candidate: Transformations such as filter and select create a new dataset definition. Actions such as count, collect and write trigger execution. Understanding this helps avoid accidental repeated computation. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Why avoid select *?

Candidate: It reads and carries columns that may not be required. Selecting only needed columns can reduce I/O, memory usage and network transfer, especially with wide datasets. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What causes a shuffle?

Candidate: Operations such as joins, groupBy and repartition may require data to move between partitions. Large shuffles can create network and disk overhead and often become a major performance bottleneck. I would support that decision with evidence from the actual workload, such as execution-plan information, Spark UI metrics, row counts or runtime comparisons. After the change, I would verify

correctness as well as performance.

Interviewer: How do you optimize a join?

Candidate: I first inspect table sizes and key distribution. I project only required columns, filter early, use an appropriate join strategy and consider broadcasting a genuinely small dimension. For skewed keys, I may need a skew-handling strategy. I would support that decision with evidence from the actual workload, such as execution-plan information, Spark UI metrics, row counts or runtime comparisons. After the change, I would verify correctness as well as performance.

Interviewer: repartition vs coalesce?

Candidate: repartition performs a shuffle and can increase or decrease partitions. coalesce generally reduces partitions with less movement. I choose based on whether I need redistribution or simply fewer output partitions. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you handle small files?

Candidate: Too many small files increase metadata and file-open overhead. I use appropriate write strategies and compaction mechanisms, while avoiding over-partitioning. The correct approach depends on workload and table characteristics. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What is partition pruning?

Candidate: If a query filters on a partitioning column, the engine can avoid scanning unrelated partitions. This reduces the amount of data read, so partitioning should align with common selective access patterns. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you choose partition columns?

Candidate: I consider query patterns, cardinality, data distribution and write behavior. I avoid very high-cardinality columns that create too many tiny partitions. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Explain Delta MERGE.

Candidate: MERGE matches source and target rows using a condition and can apply updates, inserts and, where configured, deletes. It is useful for incremental upserts when the business key and change semantics are well defined. For large tables, I would also reduce the target search space where the workload allows it rather than scanning the entire table unnecessarily. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How would you optimize a slow MERGE?

Candidate: I would first reduce the source to the required batch and project only needed columns. Then I would reduce the target search space with known predicates where valid, inspect file layout and small-file behavior, and check shuffle and concurrency characteristics. I would measure before and after because the best optimization depends on the table and workload. I would support that decision with evidence from the actual workload, such as execution-plan information, Spark UI metrics, row counts or runtime comparisons. After the change, I would verify correctness as well as performance.

Interviewer: What is time travel?

Candidate: Delta table history allows me to query or restore earlier table versions when the required history is still retained. It is useful for auditing and recovery, but it is not a substitute for a long-term backup or archival strategy. The available time-travel window depends on data and log retention settings. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Schema evolution?

Candidate: Schema evolution is not a universal automatic conversion of any schema change. In Azure Databricks, behavior depends on the component and configuration. Adding compatible columns can be supported when configured, while incompatible type changes or breaking changes may require an explicit schema change or a full rewrite. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you handle corrupt records?

Candidate: I separate malformed records from valid processing when the source and format support it. I retain enough metadata to investigate the rejected records rather than silently dropping them. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the

definition.

Interviewer: What causes driver OOM?

Candidate: The driver can run out of memory if too much data is collected to it or if driver-side objects become excessively large. I avoid collect and toPandas on large datasets and keep distributed operations distributed. I would support that decision with evidence from the actual workload, such as execution-plan information, Spark UI metrics, row counts or runtime comparisons. After the change, I would verify correctness as well as performance.

Interviewer: When is caching useful?

Candidate: Caching can help when the same expensive dataset is reused multiple times. I don't cache everything because memory is limited and unnecessary caching can increase pressure and reduce performance. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What is data skew?

Candidate: Skew occurs when some partition keys contain far more records than others. One or a few tasks then take much longer than the rest. I detect it through Spark metrics and use appropriate skew-handling techniques. I would support that decision with evidence from the actual workload, such as execution-plan information, Spark UI metrics, row counts or runtime comparisons. After the change, I would verify correctness as well as performance.

Interviewer: What is AQE?

Candidate: Adaptive Query Execution allows Spark to adapt parts of the physical plan based on runtime statistics. Depending on the workload, it can help with partition coalescing, skew handling and join strategy decisions. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you implement data quality in PySpark?

Candidate: I define explicit rules for required columns, uniqueness, ranges, referential relationships and business constraints. I record pass/fail metrics and route invalid data according to the agreed business process. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What makes a notebook production-ready?

Candidate: It should be parameterized, idempotent, observable, tested and safe to rerun. It should avoid hardcoded secrets and paths, use reusable functions where appropriate, and produce meaningful logs and metrics. I would separate immediate recovery from permanent prevention: first restore the expected service or data safely, then document the root cause, contributing factors and preventive action.

Interviewer: Can you directly edit Delta files?

Candidate: No. I would use supported Delta operations rather than manually adding, deleting or modifying the underlying Parquet files. Delta relies on its transaction log, and manual file changes can lead to lost data or inconsistent table state. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: When would you use OPTIMIZE?

Candidate: I use table-maintenance features when the workload benefits from improved file layout or compaction, but I do not treat OPTIMIZE as a universal fix. Current Databricks capabilities also include automatic file-size optimization and predictive optimization for eligible managed tables, so the maintenance strategy should match the table type, workload and platform configuration. I would support that decision with evidence from the actual workload, such as execution-plan information, Spark UI metrics, row counts or runtime comparisons. After the change, I would verify correctness as well as performance.

Interviewer: Give me your Spark performance troubleshooting framework.

Candidate: I start with evidence: Spark UI, input size, stage duration, shuffle, skew, task distribution and storage scans. Then I optimize the largest bottleneck, rerun, compare metrics and document the result. Natural transition into closing I would support that decision with evidence from the actual workload, such as execution-plan information, Spark UI metrics, row counts or runtime comparisons. After the change, I would verify correctness as well as performance.

Interviewer: Good. I think we have covered the main technical areas. Let's wrap up with a couple of questions from you. Interviewer: Alright, that covers my technical questions. Before we wrap up, do you have any questions for me?

Candidate: Yes, I have a couple of questions. Could you please tell me a little about the team I would be working with and the type of Azure data engineering challenges the team is currently handling? In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Real Interview Closing

Interviewer: Alright, that covers my technical questions. Before we wrap up, do you have any questions for me?

Candidate: Yes, I have a couple. Could you tell me a little about the team I would be working with and the main Azure data-engineering challenges the team is currently solving?

Interviewer: Sure. The team works across data platform development, production support and new data initiatives, with a focus on reliability, quality and scalability.

Candidate: Thank you. One more question: what would you expect from someone joining this role during the first three to six months?

Interviewer: We would expect the person to understand the platform, take ownership of assigned work, contribute to production support and gradually take responsibility for larger technical problems.

Candidate: That sounds good. Thank you for your time. I really enjoyed discussing the architecture, project decisions and production scenarios with you.

Interviewer: Thank you for your time as well. We'll get back to you regarding the next steps.

Interviewer checklist

✓ Project clarity • ✓ Technical reasoning • ✓ Production thinking • ✓ Troubleshooting • ✓ Performance • ✓ Security • ✓ Communication • ✓ Candidate questions

Interview 4

SQL + Data Modeling + Performance

Target experience: 4–7 years • Approx. 1 hour

Interviewer: How would you get the latest record per customer?

Candidate: I would use ROW_NUMBER partitioned by customer and ordered by the business timestamp descending, then keep row number one. I would include a deterministic tie-breaker if timestamps can be equal. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: WHERE vs HAVING?

Candidate: WHERE filters rows before aggregation. HAVING filters grouped results after aggregation. Using WHERE for row-level filters can reduce the data that needs to be aggregated. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Why can a join create duplicates?

Candidate: Usually because the join relationship is one-to-many or many-to-many when the query assumes one-to-one. I check the grain and uniqueness of join keys before changing the SQL. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What is grain?

Candidate: Grain defines what one row represents. For example, one row per order line is different from one row per order. Defining grain prevents incorrect joins and double counting. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Fact vs dimension?

Candidate: A fact table usually stores measurable business events, while dimensions store descriptive attributes used to analyze those events. The model should have a clearly defined grain. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Why star schema?

Candidate: A star schema separates facts from descriptive dimensions and is generally easy for analytical queries and reporting tools to understand. It can also simplify common aggregations. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Explain SCD Type 1 and Type 2.

Candidate: Type 1 overwrites the previous attribute value, so history is not preserved. Type 2 creates a new version of the record and retains historical values, usually with effective dates and a current indicator. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How would you implement SCD Type 2?

Candidate: I compare incoming records to the current dimension version. When a tracked attribute changes, I close the old version by setting its end date and current flag, then insert a new version with the new attributes and start date. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: A SQL query is slow. What do you do?

Candidate: I inspect the execution plan, row counts, join strategy, filters, scans and expensive operators. I check whether predicates are selective, whether joins are multiplying rows and whether statistics or physical design need attention. I would support that decision with evidence from the actual workload, such as execution-plan information, Spark UI metrics, row counts or runtime comparisons. After the change, I would verify correctness as well as performance.

Interviewer: What is a CTE?

Candidate: A common table expression gives a named query expression that can make complex SQL easier to structure and read. I use it for clarity, but I still inspect the final execution plan because readability does not automatically mean performance. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you find records in source but not target?

Candidate: An anti-join pattern such as NOT EXISTS is often appropriate. I compare on the correct business key and handle NULL semantics carefully. In a real interview, I would then give a short project example and explain what I personally owned. I

would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How would you compare very large tables?

Candidate: I avoid blindly doing expensive full comparisons. I first compare partitions, counts and key ranges, then use targeted checks or hashes where appropriate. The method depends on the business requirement for completeness. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do NULLs affect comparisons?

Candidate: NULL represents an unknown or missing value and does not behave like an ordinary value in equality comparisons. I use IS NULL/IS NOT NULL and functions such as COALESCE only when the business meaning supports it. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How can COALESCE help?

Candidate: It returns the first non-null expression and can provide a controlled fallback. I use it only when substituting a value is semantically correct, because turning unknown data into a default can hide data-quality issues. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you prevent double counting?

Candidate: I establish the grain first, then validate the cardinality of joins. If a dimension join can produce multiple matches, I resolve that ambiguity before aggregating. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What is indexing used for?

Candidate: In database systems that support indexes, indexes can accelerate selective lookups and joins by providing an access structure. They also add storage and write-maintenance overhead, so I choose them based on workload. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you optimize SQL?

Candidate: I start with the execution plan and actual workload characteristics. I reduce unnecessary rows and columns, improve predicates and joins, verify physical design and then measure the change. I would support that decision with evidence from the actual workload, such as execution-plan information, Spark UI metrics, row counts or runtime comparisons. After the change, I would verify correctness as well as performance.

Interviewer: How do you validate an aggregation?

Candidate: I compare totals at multiple levels and reconcile them against a trusted source. I also check for duplicate joins and unexpected NULL handling because a technically valid query can still produce incorrect business results. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What is a surrogate key?

Candidate: It is a system-generated identifier used to identify a dimension row independently of the source business key. It is particularly useful for tracking multiple historical versions of a business entity. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Business key vs surrogate key?

Candidate: A business key comes from the business domain, such as a customer number. A surrogate key is generated by the warehouse or platform. I preserve the business key for traceability while using the surrogate key where the dimensional model requires it. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Why use window functions?

Candidate: They allow calculations across related rows without collapsing the result set like GROUP BY does. They are useful for ranking, latest-record selection, running totals and change comparisons. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you protect dynamic SQL?

Candidate: I use parameterized queries or safe query construction rather than concatenating untrusted input. I also restrict permissions so the executing identity has only the access it needs. In a real interview, I would then give a short project

example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Give me your SQL performance workflow.

Candidate: Understand the query and expected result, inspect the execution plan, identify the largest cost, make one targeted change, test correctness, measure performance and document the improvement. I would support that decision with evidence from the actual workload, such as execution-plan information, Spark UI metrics, row counts or runtime comparisons. After the change, I would verify correctness as well as performance.

Interviewer: Tell me about a production SQL issue you solved.

Candidate: I would describe the symptom, business impact, investigation, root cause, fix and prevention. A strong answer includes measurable evidence such as runtime reduction or restored SLA, rather than only saying that the query became faster. Natural transition into closing I would separate immediate recovery from permanent prevention: first restore the expected service or data safely, then document the root cause, contributing factors and preventive action.

Interviewer: Good. I think we have covered the main technical areas. Let's wrap up with a couple of questions from you. Interviewer: Alright, that covers my technical questions. Before we wrap up, do you have any questions for me?

Candidate: Yes, I have a couple of questions. Could you please tell me a little about the team I would be working with and the type of Azure data engineering challenges the team is currently handling? In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Real Interview Closing

Interviewer: Alright, that covers my technical questions. Before we wrap up, do you have any questions for me?

Candidate: Yes, I have a couple. Could you tell me a little about the team I would be working with and the main Azure data-engineering challenges the team is currently solving?

Interviewer: Sure. The team works across data platform development, production support and new data initiatives, with a focus on reliability, quality and scalability.

Candidate: Thank you. One more question: what would you expect from someone joining this role during the first three to six months?

Interviewer: We would expect the person to understand the platform, take ownership of assigned work, contribute to production support and gradually take responsibility for larger technical problems.

Candidate: That sounds good. Thank you for your time. I really enjoyed discussing the architecture, project decisions and production scenarios with you.

Interviewer: Thank you for your time as well. We'll get back to you regarding the next steps.

Interviewer checklist

✓ Project clarity • ✓ Technical reasoning • ✓ Production thinking • ✓ Troubleshooting • ✓ Performance • ✓ Security • ✓ Communication • ✓ Candidate questions

Interview 5

Production Support + Troubleshooting

Target experience: 5-8 years • Approx. 1 hour

Interviewer: A critical pipeline fails at 2 AM. What do you do?

Candidate: First I confirm the impact and scope. I inspect the failed activity and logs, determine whether it is transient or data-related, and check downstream dependencies. If a safe retry can resolve it, I retry; otherwise I follow the incident process and communicate the impact. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Pipeline says success but target count is much lower. What next?

Candidate: I treat success status and data correctness separately. I compare source and target counts, inspect filters and incremental boundaries, verify the expected batch window and check whether late-arriving data or source changes explain the difference. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Databricks job gets OOM. What do you investigate?

Candidate: I inspect whether data is being collected to the driver, whether partitions are too large, whether joins create huge intermediate results and whether skew exists. I optimize the data flow before simply increasing cluster memory. I would support that decision with evidence from the actual workload, such as execution-plan information, Spark UI metrics, row counts or runtime comparisons. After the change, I would verify correctness as well as performance.

Interviewer: ADF Copy Activity times out. What do you check?

Candidate: I check source query duration, target write performance, network connectivity, file count, connector settings and volume. I compare the failing run with previous successful runs to identify what changed. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: API starts returning 429. What do you do?

Candidate: I reduce request pressure, use retry with backoff and respect the provider's rate-limit guidance. I avoid unlimited retries because that can worsen the problem. I would separate immediate recovery from permanent prevention: first restore the expected service or data safely, then document the root cause, contributing factors and preventive action.

Interviewer: Duplicates appear after a retry. Why?

Candidate: The load likely was not idempotent. I check whether the first attempt partially committed and whether the retry blindly appended the same batch. I would redesign the target write or batch tracking so retries converge to one correct state. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Source schema suddenly changes. What do you do?

Candidate: I determine whether the change is compatible, identify affected downstream consumers and stop or quarantine the load if continuing could corrupt data. Then I update the contract and deployment through controlled testing. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you determine incident severity?

Candidate: I use business impact, affected users or processes, data integrity, SLA impact and duration. The exact severity definitions should follow the organization's incident policy. I would separate immediate recovery from permanent prevention: first restore the expected service or data safely, then document the root cause, contributing factors and preventive action.

Interviewer: What is RCA?

Candidate: Root Cause Analysis identifies what happened, why it happened, why existing controls did not prevent it and what actions will reduce recurrence. It should focus on evidence and prevention rather than blame. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What should an RCA contain?

Candidate: Incident summary, timeline, impact, detection, technical root cause, contributing factors, recovery, corrective actions, owners and target dates. It should also identify monitoring or process improvements. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover

implementation as well as the definition.

Interviewer: How do you monitor data quality?

Candidate: I monitor completeness, validity, uniqueness, consistency and business-specific rules. I publish metrics and alerts so failures are visible before incorrect data reaches consumers. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you monitor freshness?

Candidate: I measure the age of the latest successfully processed business data against the expected SLA. I alert when the freshness threshold is exceeded, not merely when a pipeline fails. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: A zero-byte file arrives. What do you do?

Candidate: I don't automatically treat it as valid input. I check whether an empty file is expected, verify source behavior and route it according to the agreed contract. Unexpected empty files should trigger an operational signal. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What about a partially written file?

Candidate: I need a reliable completion signal such as atomic file creation, event semantics or a source-provided marker. Processing a file while it is still being written can result in incomplete data. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you safely rerun production?

Candidate: I first determine what the failed run actually committed and identify the exact batch or time window. Then I use an idempotent write pattern, or clean up the known partial state before rerunning, and validate counts, keys and business rules afterward. I never rerun blindly just because the pipeline is marked failed. I would separate immediate recovery from permanent prevention: first restore the expected service or data safely, then document the root cause, contributing factors and preventive action.

Interviewer: How do you communicate during an incident?

Candidate: I provide concise updates covering impact, current status, what is being investigated, workaround or recovery plan and the next update time. I avoid guessing and clearly separate confirmed facts from hypotheses. I would separate immediate recovery from permanent prevention: first restore the expected service or data safely, then document the root cause, contributing factors and preventive action.

Interviewer: Deployment fails in production. What do you check?

Candidate: I check the deployment logs, artifact version, environment configuration, permissions, dependencies and parameter values. If necessary I follow the rollback procedure rather than making untracked manual changes. I would separate immediate recovery from permanent prevention: first restore the expected service or data safely, then document the root cause, contributing factors and preventive action.

Interviewer: How do you reduce recurring incidents?

Candidate: I identify patterns across incidents and prioritize permanent fixes such as better validation, observability, retry logic, dependency controls or automation. The objective is to reduce both frequency and recovery time. I would separate immediate recovery from permanent prevention: first restore the expected service or data safely, then document the root cause, contributing factors and preventive action.

Interviewer: What logs are most useful?

Candidate: I want run ID, batch ID, timestamps, source and target, parameters, row counts, status, error details and dependency information. Correlation IDs are especially useful when several services are involved. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What is observability in a data platform?

Candidate: It means being able to understand system behavior from logs, metrics and traces. For data platforms, observability also includes freshness, volume, quality and lineage signals. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Dev works but production fails. What do you compare?

Candidate: I compare deployed versions, configuration, identities, network access, secret references, data volume, source behavior and environment-specific parameters. I avoid assuming the code is the only difference. I would separate immediate recovery from permanent prevention: first restore the expected service or data safely, then document the root cause, contributing factors and preventive

action.

Interviewer: How would you handle bad data already loaded?

Candidate: I first stop downstream propagation if necessary, identify the affected batch and determine the safest correction or rollback path. I preserve evidence for investigation and validate the corrected state before reopening downstream consumption. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you prevent bad data from reaching reports?

Candidate: I add quality gates before publishing trusted datasets, separate quarantine from curated data and make critical checks visible to consumers. A pipeline should not be considered successful simply because a compute job completed. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What does mature production support look like?

Candidate: It combines reliable design, monitoring, clear runbooks, controlled deployments, automated validation, incident discipline and continuous improvement. The goal is to reduce dependency on manual heroics. Natural transition into closing I would separate immediate recovery from permanent prevention: first restore the expected service or data safely, then document the root cause, contributing factors and preventive action.

Interviewer: Good. I think we have covered the main technical areas. Let's wrap up with a couple of questions from you. Interviewer: Alright, that covers my technical questions. Before we wrap up, do you have any questions for me?

Candidate: Yes, I have a couple of questions. Could you please tell me a little about the team I would be working with and the type of Azure data engineering challenges the team is currently handling? In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Real Interview Closing

Interviewer: Alright, that covers my technical questions. Before we wrap up, do you have any questions for me?

Candidate: Yes, I have a couple. Could you tell me a little about the team I would be working with and the main Azure data-engineering challenges the team is currently solving?

Interviewer: Sure. The team works across data platform development, production support and new data initiatives, with a focus on reliability, quality and scalability.

Candidate: Thank you. One more question: what would you expect from someone joining this role during the first three to six months?

Interviewer: We would expect the person to understand the platform, take ownership of assigned work, contribute to production support and gradually take responsibility for larger technical problems.

Candidate: That sounds good. Thank you for your time. I really enjoyed discussing the architecture, project decisions and production scenarios with you.

Interviewer: Thank you for your time as well. We'll get back to you regarding the next steps.

Interviewer checklist

✓ Project clarity • ✓ Technical reasoning • ✓ Production thinking • ✓ Troubleshooting • ✓ Performance • ✓ Security • ✓ Communication • ✓ Candidate questions

Interview 6

End-to-End Project + Architecture

Target experience: 5-8 years • Approx. 1 hour

Interviewer: Suppose I ask you to design a customer analytics platform. Where do you start?

Candidate: I start with requirements: sources, volume, frequency, latency, consumers, retention, security and data-quality expectations. Then I define the logical architecture and choose services based on those requirements. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What architecture would you propose?

Candidate: A common Azure architecture is source systems → ADF ingestion → ADLS raw → Databricks transformation → Delta curated/serving → reporting or downstream consumers. The exact components depend on latency, scale and governance requirements. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Batch or streaming?

Candidate: I choose based on business latency rather than technology preference. If hourly or daily data is sufficient, batch can be simpler and cheaper. If consumers require near-real-time data, a streaming architecture may be justified. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you secure the platform?

Candidate: Use managed identities where possible, least-privilege RBAC/ACLs, Key Vault for secrets, private connectivity where required, encryption and environment separation. Sensitive data should also have appropriate governance and access controls. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you separate environments?

Candidate: I use separate development, UAT and production resources or controlled resource boundaries. Configuration is externalized so the same deployment artifact can be promoted with environment-specific values. In a real interview, I would then

give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How would you manage source control?

Candidate: I keep code and pipeline definitions in a version-control system with branches, pull requests and review. Deployment should be traceable to a specific commit or artifact. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you implement ADF CI/CD?

Candidate: I keep the development factory connected to Git, use feature branches and pull requests, publish the approved development state, and promote it to UAT and production through CI/CD with environment-specific parameters. Production should not become a manually edited copy of development. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you manage configuration?

Candidate: I separate code from configuration such as storage paths, database endpoints and feature flags. Secrets are stored securely and identities are used where possible. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you provide lineage?

Candidate: I document source-to-target mappings and use available catalog or lineage capabilities where appropriate. At minimum, every curated dataset should have a clear source, transformation and ownership trail. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you plan capacity?

Candidate: I estimate data volume, concurrency, transformation complexity and SLA. I start from measured workload characteristics and then tune compute, parallelism and storage layout. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you design for failure?

Candidate: I assume source failures, network errors, compute failures and bad data will happen. I use retries for transient problems, checkpoints or batch state, idempotent writes, validation and clear recovery procedures. I would separate immediate recovery from permanent prevention: first restore the expected service or data safely, then document the root cause, contributing factors and preventive action.

Interviewer: How would you build reusable ingestion?

Candidate: I define metadata for source, target, load pattern, keys and incremental logic, then build a generic orchestration framework. New tables should require configuration rather than a new pipeline wherever the pattern is the same. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What is a data contract?

Candidate: It is an agreed expectation between producers and consumers covering schema, meaning, quality, freshness and change rules. It makes breaking changes visible and reduces surprises downstream. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What quality gates would you add?

Candidate: Completeness, uniqueness, referential integrity, accepted ranges, schema validation and freshness are common examples. Critical failures should prevent publishing trusted data. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How would you handle a privacy deletion request?

Candidate: I first identify where the subject's data exists, including curated and downstream stores. Then I follow the organization's approved deletion and audit process and validate that required systems are updated. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you document architecture?

Candidate: I document context, requirements, components, data flows, security, failure handling, operational ownership and key tradeoffs. The document should help another engineer operate and evolve the platform. In a real interview, I would then

give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you measure success?

Candidate: I use business and engineering metrics such as SLA attainment, freshness, data-quality pass rate, pipeline failure rate, recovery time, cost and consumer adoption. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Requirements change halfway through. What do you do?

Candidate: I clarify the new requirement, assess impact on architecture, schedule, cost and downstream consumers, then propose options with explicit tradeoffs. I avoid silently changing scope. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you conduct an architecture review?

Candidate: I review requirements, scale assumptions, security, reliability, performance, cost, operability and future change. I want stakeholders to understand both the design and the reasons behind the major choices. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you control cost?

Candidate: I measure compute and storage usage, optimize inefficient workloads, manage file layout, use appropriate scheduling and avoid unnecessary always-on resources. Cost optimization should not compromise required SLAs or data quality. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you protect CI/CD secrets?

Candidate: Secrets should not be committed to source control. I use managed identity or approved secret-management services and limit deployment permissions. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: UAT works but production behaves differently. Why?

Candidate: Environment configuration, permissions, data volume, network access, identities, dependencies and parameter values can differ. I compare those systematically rather than assuming the code changed. I would separate immediate recovery from permanent prevention: first restore the expected service or data safely, then document the root cause, contributing factors and preventive action.

Interviewer: How would you migrate an on-prem SQL Server workload?

Candidate: I inventory tables and dependencies, classify full versus incremental loads, design landing and transformation patterns, validate data reconciliation and run parallel testing before cutover. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How would you present the project to an interviewer?

Candidate: I use a structured story: business problem, architecture, my responsibilities, difficult technical decisions, production challenges, measurable improvements and lessons learned. I keep enough detail to invite technical follow-up. Natural transition into closing In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Good. I think we have covered the main technical areas.

Let's wrap up with a couple of questions from you. Interviewer: Alright, that covers my technical questions. Before we wrap up, do you have any questions for me?

Candidate: Yes, I have a couple of questions. Could you please tell me a little about the team I would be working with and the type of Azure data engineering challenges the team is currently handling? In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Real Interview Closing

Interviewer: Alright, that covers my technical questions. Before we wrap up, do you have any questions for me?

Candidate: Yes, I have a couple. Could you tell me a little about the team I would be working with and the main Azure data-engineering challenges the team is currently solving?

Interviewer: Sure. The team works across data platform development, production support and new data initiatives, with a focus on reliability,

quality and scalability.

Candidate: Thank you. One more question: what would you expect from someone joining this role during the first three to six months?

Interviewer: We would expect the person to understand the platform, take ownership of assigned work, contribute to production support and gradually take responsibility for larger technical problems.

Candidate: That sounds good. Thank you for your time. I really enjoyed discussing the architecture, project decisions and production scenarios with you.

Interviewer: Thank you for your time as well. We'll get back to you regarding the next steps.

Interviewer checklist

✓ Project clarity • ✓ Technical reasoning • ✓ Production thinking • ✓ Troubleshooting • ✓ Performance • ✓ Security • ✓ Communication • ✓ Candidate questions

Interview 7

Senior Final Mock Interview

Target experience: 6–10 years • Approx. 1 hour

Interviewer: Design a platform that processes 10 TB of data per day. How would you approach it?

Candidate: I would start with ingestion patterns, source throughput, SLA, concurrency, data retention, consumers and security. I would design scalable object storage, distributed processing, partition-aware data layouts, resilient orchestration and monitoring. I would validate the architecture with realistic volume and failure tests. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you design resilience?

Candidate: I identify failure domains and make each stage restartable. I use retries for transient failures, durable state, idempotent writes, validation, alerting and recovery runbooks. Resilience also means avoiding a single component becoming an unnecessary point of failure. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What is a common architecture mistake?

Candidate: One common mistake is designing only for the happy path. Production systems need explicit handling for retries, duplicates, late data, schema changes, partial failures, security and observability. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: A large join is extremely skewed. What do you do?

Candidate: I confirm skew using execution metrics and key-frequency analysis. Depending on the case, I can change the join strategy, isolate heavy keys, use salting or other supported skew-handling techniques and validate the effect. I would support that decision with evidence from the actual workload, such as execution-plan information, Spark UI metrics, row counts or runtime comparisons. After the change, I would verify correctness as well as performance.

Interviewer: Cloud cost suddenly increases. How do you investigate?

Candidate: I compare cost by service, workload and time period, then correlate the increase with deployments, data volume, cluster size, job frequency or storage growth. I fix the actual driver rather than applying a blanket reduction. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: When would you choose a managed service over custom code?

Candidate: I prefer managed services when they meet requirements for reliability, scale, security and operations because they reduce maintenance. Custom components are justified when there is a real requirement the managed option cannot satisfy. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How would you migrate legacy ETL?

Candidate: I inventory existing workflows, dependencies and business rules, identify high-risk logic and define target architecture. I migrate incrementally, reconcile outputs and run parallel validation before retiring legacy processing. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you manage technical debt?

Candidate: I make debt visible, quantify its operational impact and prioritize it alongside feature work. Critical reliability or security debt gets higher priority than cosmetic refactoring. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you mentor junior engineers?

Candidate: I explain the reasoning behind designs, review code constructively and give progressively larger ownership. I also encourage them to troubleshoot and propose solutions rather than simply providing answers. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What if an architect disagrees with your design?

Candidate: I focus on requirements, evidence and tradeoffs rather than personal preference. I explain my reasoning, listen to the alternative and agree on a decision using the team's architecture process. In a real interview, I would then give a short

project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you enforce a data contract?

Candidate: I define ownership, schema expectations, quality rules, freshness and change-management procedures. Automated validation and communication around breaking changes make the contract operational rather than just a document. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What do you do with a breaking schema change?

Candidate: I identify impacted consumers, pause or quarantine unsafe processing if needed, coordinate the contract change and deploy compatible updates. I avoid silently coercing incompatible data just to make a pipeline appear successful. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Exactly-once or effectively-once — how do you explain the difference?

Candidate: Exactly-once describes a stronger processing guarantee that can be difficult to maintain across distributed systems and external side effects. In many practical data platforms, we design for an effectively-once outcome: retries may occur, but durable state, deterministic batch IDs, idempotent writes and deduplication make the final business result equivalent to one successful processing. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How would you replay data?

Candidate: I keep immutable or recoverable raw data and maintain batch or event boundaries so a historical window can be reprocessed. The replay path should not corrupt the current target and should be validated before publishing. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Micro-batch or streaming?

Candidate: I choose based on latency requirements, source behavior, operational complexity and cost. Micro-batch can satisfy near-real-time needs with simpler operations in many cases, while true streaming is appropriate when continuous

low-latency processing is required. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: How do you detect silent data loss?

Candidate: I monitor expected versus actual volumes, freshness, control totals, key coverage and business-level reconciliation. Successful job status alone is not enough to prove completeness. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What does governance mean for a data platform?

Candidate: It includes ownership, access control, classification, lineage, retention, quality, privacy and change management. Governance should be built into platform processes rather than treated as a separate document. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Performance or cost — which do you prioritize?

Candidate: I start with the agreed business SLA and then optimize cost within that requirement. If performance is already well inside the SLA, extra compute may not provide business value. I would support that decision with evidence from the actual workload, such as execution-plan information, Spark UI metrics, row counts or runtime comparisons. After the change, I would verify correctness as well as performance.

Interviewer: When would you reject a proposed design?

Candidate: If it cannot meet a critical requirement for security, reliability, scale, data quality or operational support, I would explain the gap and propose alternatives. The decision should be based on requirements and evidence. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Tell me about an incident you led.

Candidate: I would structure it around impact, detection, triage, coordination, recovery, root cause and prevention. At senior level, I would emphasize how I kept the response organized and improved the platform afterward. I would separate immediate recovery from permanent prevention: first restore the expected service or data safely, then document the root cause, contributing factors and preventive action.

Interviewer: How do you prove reliability?

Candidate: I use measurable evidence such as successful-run rate, SLA attainment, freshness, quality pass rate, recovery time and incident trends. I also validate failure and recovery paths through testing. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Tell me about a platform improvement using STAR.

Candidate: I would describe the Situation and business impact, the Task I owned, the Actions I took with technical detail, and the measurable Result. I would include what I learned and how I prevented recurrence. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What would you do in your first 30 days in a senior role?

Candidate: I would learn the business and architecture, understand production pain points, review operational metrics and build relationships with the team. I would avoid making major changes before understanding the system and its constraints. In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: What makes you ready for a senior Azure Data Engineer role?

Candidate: I would demonstrate ownership beyond writing pipelines: architecture decisions, production troubleshooting, performance and cost optimization, security, mentoring, communication and measurable delivery outcomes. Natural transition into closing In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Interviewer: Good. I think we have covered the main technical areas. Let's wrap up with a couple of questions from you. Interviewer: Alright, that covers my technical questions. Before we wrap up, do you have any questions for me?

Candidate: Yes, I have a couple of questions. Could you please tell me a little about the team I would be working with and the type of Azure data engineering challenges the team is currently handling? In a real interview, I would then give a short project example and explain what I personally owned. I would also mention how I validated the result, because a strong answer should cover implementation as well as the definition.

Real Interview Closing

Interviewer: Alright, that covers my technical questions. Before we wrap up, do you have any questions for me?

Candidate: Yes, I have a couple. Could you tell me a little about the team I would be working with and the main Azure data-engineering challenges the team is currently solving?

Interviewer: Sure. The team works across data platform development, production support and new data initiatives, with a focus on reliability, quality and scalability.

Candidate: Thank you. One more question: what would you expect from someone joining this role during the first three to six months?

Interviewer: We would expect the person to understand the platform, take ownership of assigned work, contribute to production support and gradually take responsibility for larger technical problems.

Candidate: That sounds good. Thank you for your time. I really enjoyed discussing the architecture, project decisions and production scenarios with you.

Interviewer: Thank you for your time as well. We'll get back to you regarding the next steps.

Interviewer checklist

✓ Project clarity • ✓ Technical reasoning • ✓ Production thinking • ✓ Troubleshooting • ✓ Performance • ✓ Security • ✓ Communication • ✓ Candidate questions